



Aula 13: Generics e Coleções em Java

Programação Orientada a Objetos

Nesta aula, vamos estudar como armazenar, organizar e manipular conjuntos de objetos em Java utilizando o Java Collections Framework. Também veremos o papel dos Generics, que permitem criar estruturas mais seguras e reutilizáveis, evitando conversões manuais e erros de tipo em tempo de execução.

POO EM
JAVA

AULA 13

NÍVEL INICIANTE

Objetivos de Aprendizagem

Ao final desta aula, você será capaz de:

01

Compreender Generics

Entender como o uso de `<T>` permite criar classes, métodos e estruturas reutilizáveis com segurança de tipos.

03

3. Manipular listas de objetos

Adicionar, remover, buscar e percorrer objetos armazenados em coleções.

02

Utilizar Coleções em Java

Conhecer as principais interfaces e classes do Java Collections Framework, como `List`, `ArrayList`, `Set`, `HashSet`, `Map` e `HashMap`.

04

4. Aplicar em sistemas orientados a objetos

Usar coleções em sistemas reais, como cadastro de clientes, veículos, produtos ou alunos.

Por que precisamos de Coleções?

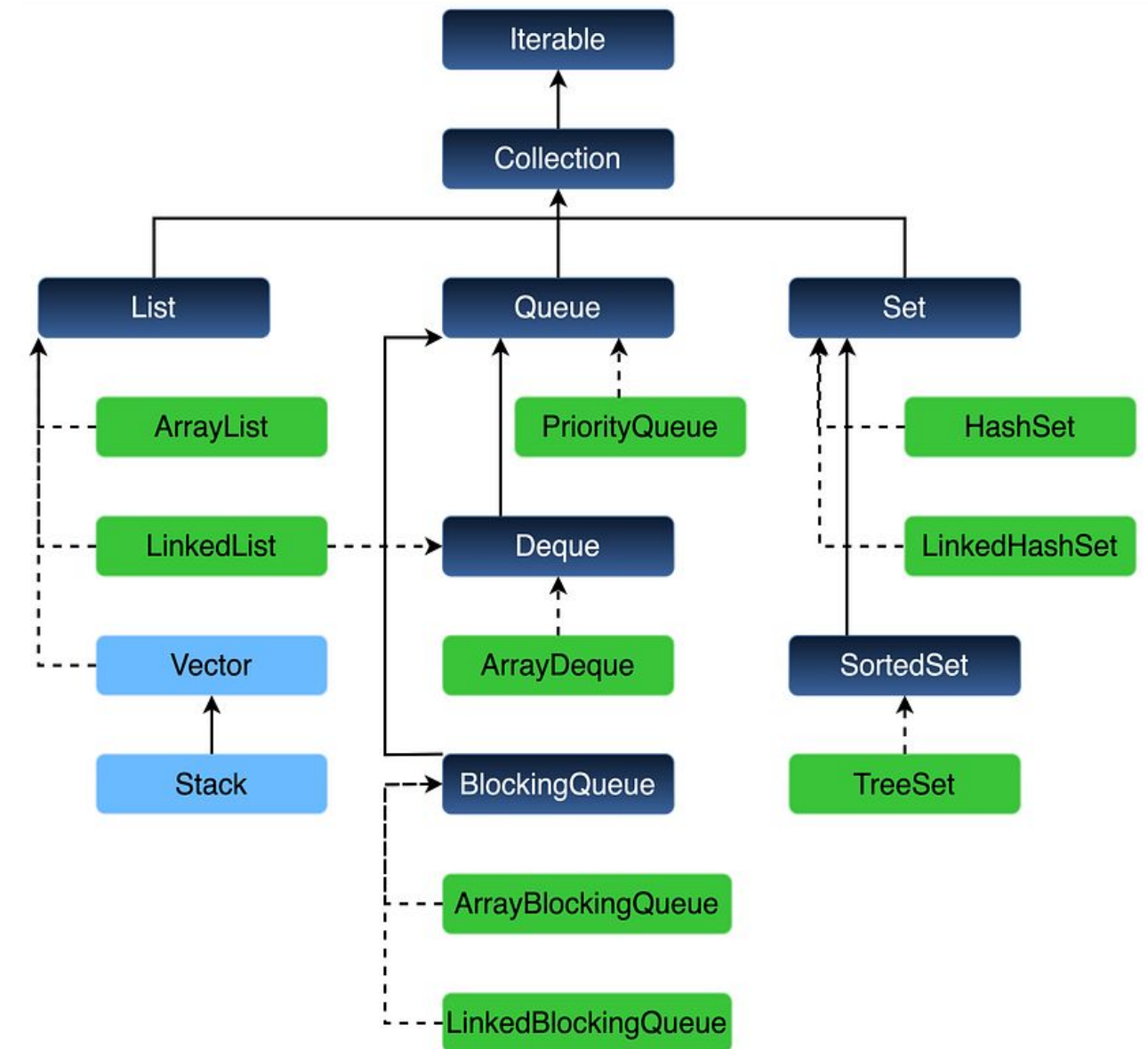
Até agora, muitos exemplos usam apenas um objeto por vez:

```
Cliente c1 = new Cliente("Maria");  
Cliente c2 = new Cliente("João");  
Cliente c3 = new Cliente("Ana");
```

Mas em um sistema real, normalmente precisamos trabalhar com vários objetos:

```
vários clientes  
vários veículos  
várias contas  
várias locações  
vários produtos
```

Para isso, Java oferece estruturas prontas chamadas coleções. Uma coleção permite armazenar e manipular vários elementos sem precisar criar dezenas de variáveis separadas.



Problema do Array Tradicional

Antes das coleções, poderíamos usar arrays:

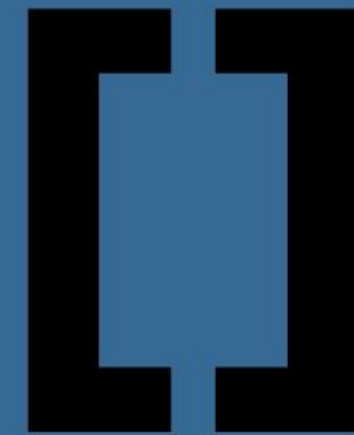
```
Cliente[] clientes = new Cliente[3];  
  
clientes[0] = new Cliente("Maria");  
clientes[1] = new Cliente("João");  
clientes[2] = new Cliente("Ana");
```

O problema é que arrays têm tamanho fixo.

Se precisar adicionar um quarto cliente, o array não aumenta automaticamente.

```
clientes[3] = new Cliente("José"); // erro
```

Além disso, remover elementos, buscar objetos e reorganizar dados pode ficar trabalhoso.



Array

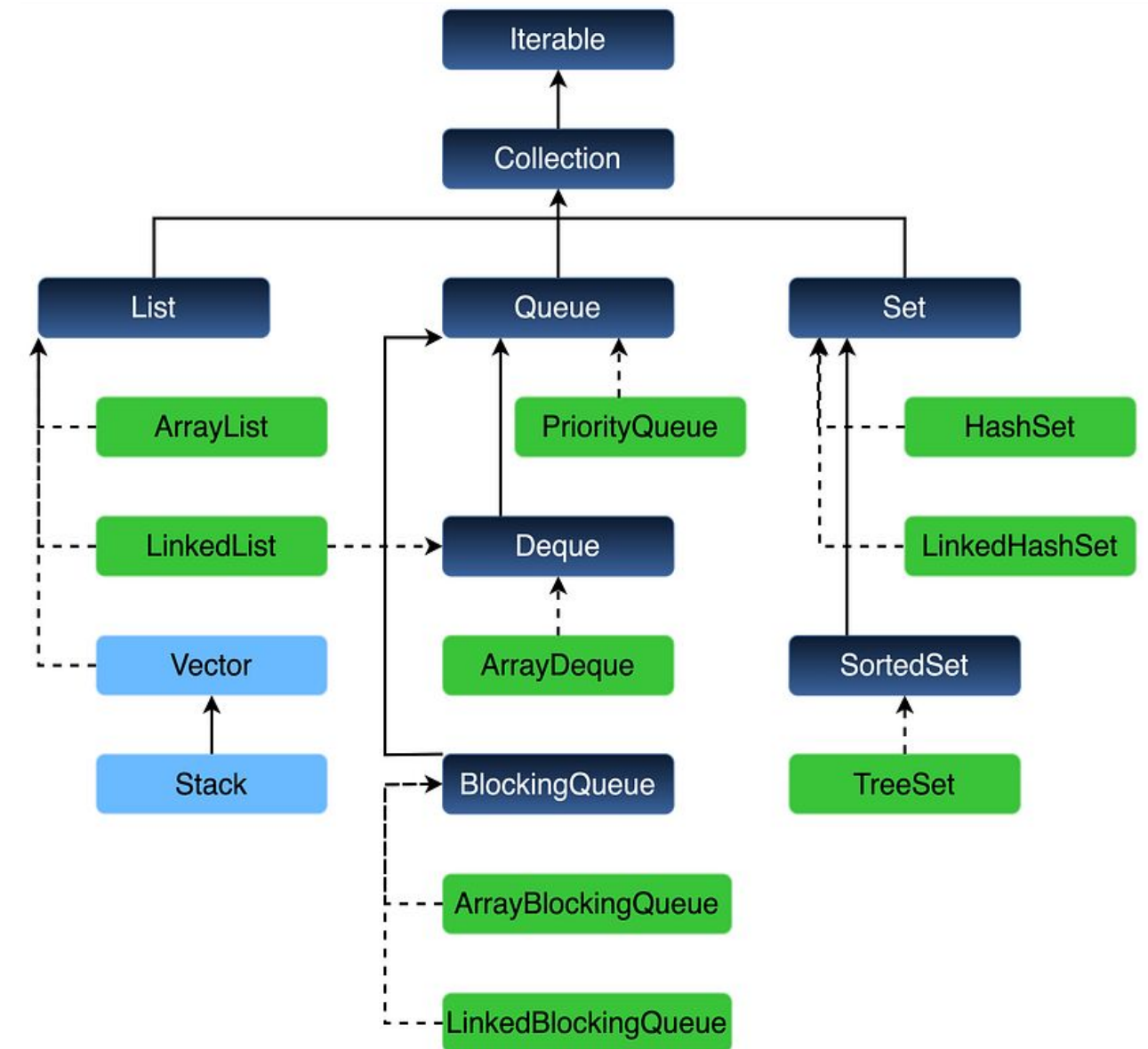
O que é o Java Collections Framework?

O Java Collections Framework é um conjunto de interfaces e classes prontas para armazenar e manipular grupos de objetos. Ele fornece estruturas como:

```
List
Set
Map
Queue
ArrayList
HashSet
HashMap
LinkedList
```

A vantagem é que essas estruturas já vêm prontas, testadas e otimizadas.

Em vez de criar tudo manualmente, usamos classes da própria linguagem.



Por que usar Generics?

Generics ajudam em três pontos principais:

Segurança de tipo

O compilador impede que você adicione um tipo errado.

```
List<String> nomes = new ArrayList<>();
```

```
nomes.add("Maria");
```

```
nomes.add(10); // erro de compilação
```

Menos casts manuais

Sem Generics, seria necessário converter objetos manualmente.

Código mais reutilizável

Você pode criar classes e métodos que funcionam com diferentes tipos.

List e ArrayList

A interface List representa uma coleção ordenada. Ela permite elementos repetidos e acesso por posição.

```
import java.util.ArrayList;
import java.util.List;

public class ExemploLista {
    public static void main(String[] args) {
        List<String> nomes = new ArrayList<>();

        nomes.add("Maria");
        nomes.add("João");
        nomes.add("Ana");

        System.out.println(nomes.get(0));
    }
}

// Saída = Maria
```


Métodos comuns de List

Alguns métodos importantes:

Função	Método
Adiciona um elemento.	add(elemento)
Remove um elemento.	remove(elemento)
Retorna o elemento de determinada posição.	get(indice)
Retorna a quantidade de elementos.	size()
Verifica se o elemento existe na lista.	contains(elemento)

Podemos percorrer uma lista usando for tradicional:

```
for (int i = 0; i < nomes.size(); i++) {  
    System.out.println(nomes.get(i));  
}
```

Ou usando for-each:

```
for (String nome : nomes) {  
    System.out.println(nome);  
}
```

O for-each é mais simples quando não precisamos do índice.

Lista de Objetos

Usando objetos próprios:

```
public class Cliente {  
    private String nome;  
    private String cpf;  
  
    public Cliente(String nome, String cpf) {  
        this.nome = nome;  
        this.cpf = cpf;  
    }  
  
    public String getNome() {  
        return nome;  
    }  
  
    public String getCpf() {  
        return cpf;  
    }  
}
```

Usando a lista:

```
List<Cliente> clientes = new ArrayList<>();  
  
clientes.add(new Cliente("Maria", "111"));  
clientes.add(new Cliente("João", "222"));  
  
for (Cliente c : clientes) {  
    System.out.println(c.getNome());  
}
```

Set e HashSet

A interface `Set` representa uma coleção que não aceita elementos repetidos.

```
import java.util.HashSet;
import java.util.Set;

public class ExemploSet {
    public static void main(String[] args) {
        Set<String> nomes = new HashSet<>();

        nomes.add("Maria");
        nomes.add("João");
        nomes.add("Maria");

        System.out.println(nomes);
    }
}
```

Mesmo adicionando “Maria” duas vezes, ela aparece apenas uma vez.

O Set é útil quando não queremos duplicidade.

Map e HashMap

A interface `Map` armazena dados no formato chave e valor.

```
import java.util.HashMap;
import java.util.Map;

public class ExemploMap {
    public static void main(String[] args) {
        Map<String, Cliente> clientes = new HashMap<>();

        Cliente c1 = new Cliente("Maria", "111");
        Cliente c2 = new Cliente("João", "222");

        clientes.put(c1.getCpf(), c1);
        clientes.put(c2.getCpf(), c2);

        Cliente encontrado = clientes.get("111");

        System.out.println(encontrado.getNome());
    }
}
```

📌 Aqui `String` é o tipo da chave. `Cliente` é o tipo do valor. `Map<TipoDaChave, TipoDoValor>`

Comparando List, Set e Map

As três estruturas armazenam coleções de dados, mas cada uma resolve um problema diferente.

List organiza elementos em sequência, Set evita duplicações e Map associa uma chave a um valor.

Estrutura	Características	Exemplo de Uso
List	Mantém ordem e aceita repetidos	Lista de alunos
Set	Não aceita repetidos	CPFs cadastrados
Map	Usa chave e valor	Buscar cliente pelo CPF

Generics em Classes Próprias

Também podemos criar nossas próprias classes genéricas.

```
public class Caixa<T> {  
    private T valor;  
  
    public void guardar(T valor) {  
        this.valor = valor;  
    }  
  
    public T pegar() {  
        return valor;  
    }  
}  
  
Caixa<String> caixaTexto = new Caixa<>();  
caixaTexto.guardar("Olá");  
  
Caixa<Integer> caixaNumero = new Caixa<>();  
caixaNumero.guardar(10);
```

Também é possível criar métodos genéricos:

```
public class Util {  
    public static <T> void imprimir(T valor) {  
        System.out.println(valor);  
    }  
}  
  
Util.imprimir("Texto");  
Util.imprimir(10);  
Util.imprimir(3.14);
```

Exemplo Prático: Cadastro de Clientes

```
import java.util.ArrayList;
import java.util.List;

public class CadastroClientes {
    private List<Cliente> clientes = new ArrayList<>();

    public void adicionar(Cliente cliente) {
        clientes.add(cliente);
    }

    public void listar() {
        for (Cliente c : clientes) {
            System.out.println(c.getNome() + " - " + c.getCpf());
        }
    }

    public Cliente buscarPorCpf(String cpf) {
        for (Cliente c : clientes) {
            if (c.getCpf().equals(cpf)) { return c; }
        }
        return null;
    }
}
```

Coleções com Exceções

Podemos combinar coleções com tratamento de exceções, conteúdo visto na Aula 12.

```
public void adicionar(Cliente cliente) {  
    if (cliente == null) {  
        throw new IllegalArgumentException("Cliente não pode ser nulo.");  
    }  
  
    clientes.add(cliente);  
}  
  
// Uso  
  
try {  
    cadastro.adicionar(null);  
} catch (IllegalArgumentException e) {  
    System.out.println("Erro: " + e.getMessage());  
}
```

📌 Assim, o sistema evita dados inválidos e trata o erro de forma controlada.

Boas Práticas

Declare usando interfaces

Prefira declarar pela interface para reduzir o acoplamento e facilitar a troca de implementação.

Exemplo: Prefira `List<Cliente> clientes = new ArrayList<>();` ao em vez de `ArrayList<Cliente> clientes = new ArrayList<>();`

Use Generics sempre que possível

Evite coleções sem tipo definido, pois isso pode gerar erros em tempo de execução e exigir conversões manuais.

Exemplo: Evite `ArrayList lista = new ArrayList();` utilize `ArrayList<> lista = new ArrayList<>();`

Escolha a coleção correta

Cada estrutura deve ser escolhida de acordo com a necessidade do problema. Use `List` quando a ordem dos elementos importa.

Use `Set` quando não pode haver repetição. Use `Map` quando precisa buscar rapidamente um valor usando uma chave.

Valide dados antes de inserir

Não deixe objetos inválidos entrarem na coleção. Antes de adicionar um elemento, verifique se ele possui os dados necessários, como nome, CPF ou outro identificador válido. Exemplo:

```
if (cliente != null && cliente.getCpf() != null) {  
    clientes.add(cliente);  
}
```